

Synthesizing Staged Updates between Verified Controllers

ANONYMOUS AUTHOR(S)

Changing several components or requirements together can remove a state needed for update completion; separating them can expose harmful behavior. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) generates the intervening game from local transfers, requirement lifetimes and initial obligations between fixed verified controllers. Winning policies keep active monitors safe, finish from every old entry under every modeled outcome without fairness, and restore the selected endpoint continuation. Our second contribution identifies when update granularity can change. A local transfer preserves global uncontrollable quiescence in every synchronous context exactly when it offers no new uncontrollable label and removes no blocker; with additional outcome, monitor and precedence conditions, this suffices to expand a winning batch policy into individual commands. A capacity theorem and analytical counterexamples delimit these conditions. Constructions separate both directions, whereas none of 54 inherited comparisons changes its decision; ten saved-policy expansions check the construction on a restricted family. An implementation using established lazy reachability-game exploration decides 26 of 27 adapted contracts within 1,200 s. The scope is finite maintenance: commands require global quiescence, and neither continuous service nor elapsed-time bounds are guaranteed.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Synthesizing Staged Updates between Verified Controllers. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

A controller update must coordinate component replacements with requirements that change at different times [3, 7, 14, 25]. Correctness of the old and new controllers does not establish a safe transition [2, 31]. Its *granularity* matters too. Replacing both Cell stations together is impossible because one holds the workpiece (Section 2); replacing two components of another model separately exposes an uncontrollable (UC) loop absent from both endpoints (Section 5.3). Neither grouping nor separating operations preserves feasibility in general.

Dynamic Update Controller Synthesis (DUCS) [30, 31] and generalized-reactivity(1), or GR(1), updating [2] already generate update control. FG-DUCS fixes both endpoint controllers. Two design questions remain after their verification: which local inputs determine the intervening game and which choices synthesis makes (C1), and when a winning batch policy can be expanded into individual commands (C2). We answer the first and give sufficient conditions and obstructions for the second, using established reachability-game synthesis.

C1: A local contract for staged handover (Sections 3–4). The modeler supplies transfer outcomes, requirement lifetimes and initial obligations; the generator supplies mixed configurations, guards and handover tests. Synthesis chooses coordination and a handover target from every reachable old state. Winners keep active monitors safe and finish under all modeled outcomes without fairness. They restore the selected endpoint’s permitted behavior, which safety requirements alone need not preserve. We distinguish this game guarantee from requirement interpretation and platform execution.

C2: Conditions for changing update granularity (Section 5). A batch removes the states between its members, changing the update contract. Given a winning batch policy, sufficient conditions construct an individual-command policy without solving the fine game again, preserving completion and the selected endpoint. An exact test characterizes UC quiescence preservation in every synchronous context using local alphabets, enabled UC labels and transfer pairs. Additional outcome, monitor and precedence conditions protect intermediate states. Obstructions and a capacity threshold rule out specified merges; failing the test leaves feasibility open.

The intended use is checking a finite maintenance plan before execution. Work may pause. Every command and handover requires global UC quiescence: no environment event that control cannot disable is enabled. The guarantee counts events, not elapsed time; persistent UC activity can prevent completion.

The evidence has distinct roles (Section 6). Constructed mechanisms and analytical counterexamples explain both granularity directions; none of 54 inherited comparisons changes its decision. These establish possibilities, not prevalence. Saved policies check serialization on one restricted family; game and source checks test selected guarantee layers; 27 adapted contracts compare exploration costs within one solver.

We created the public replication package [4] for this research. TA (Technical Appendix) and S1–S5 denote the accompanying *Supplementary Material*; Data Availability identifies its matching version and source.

2 Why Updates Need Separate Component and Requirement Choices

Cell motivates transfers and initial duties; Policy motivates separate requirement boundaries.

2.1 Cell: Moving Work before Replacing a Station

Our constructed *Cell* example has two stations A, B and exactly one workpiece [22, 31]. A station can be replaced only while empty. The old controller moves the workpiece between stations and forbids inspection. The new controller requires inspection at B before departure. Both endpoints are verified, yet simultaneous replacement is impossible: one station always holds the workpiece.

Coordinating moves and changing duties. Figure 1 shows the policy from both old states, called *entries*. Each path replaces the empty station, moves the workpiece, and later replaces the other station. The input requires new-start before old-stop, preventing an unprotected gap; synthesis chooses the remaining order. If B holds at new-start, inspection becomes pending; it must wait until old-stop removes the ban. Starting its monitor—the automaton tracking the new rule—clear would silently discard this duty.

The first path inspects before handover; the second starts with B empty and needs no inspection. Both reach allowed new closed-loop states with inspection clear: z_A holds at A , and z_B at B . A nonnegative *rank* decreases on every arrow, excluding safe but endless inspection.

What synthesis chooses. For Cell, the modeler supplies transfer relations $g_A = g_B = \{(e, e)\}$, where e means empty; new-start precedes old-stop; the new initializer installs pending iff B holds; and the loadable targets are z_A, z_B . Synthesis chooses the remaining sequence and target. All Cell events are controllable: m_A, m_B move to A, B , j inspects, and ρ_A, ρ_B replace a station. TA B gives the complete input and proof.

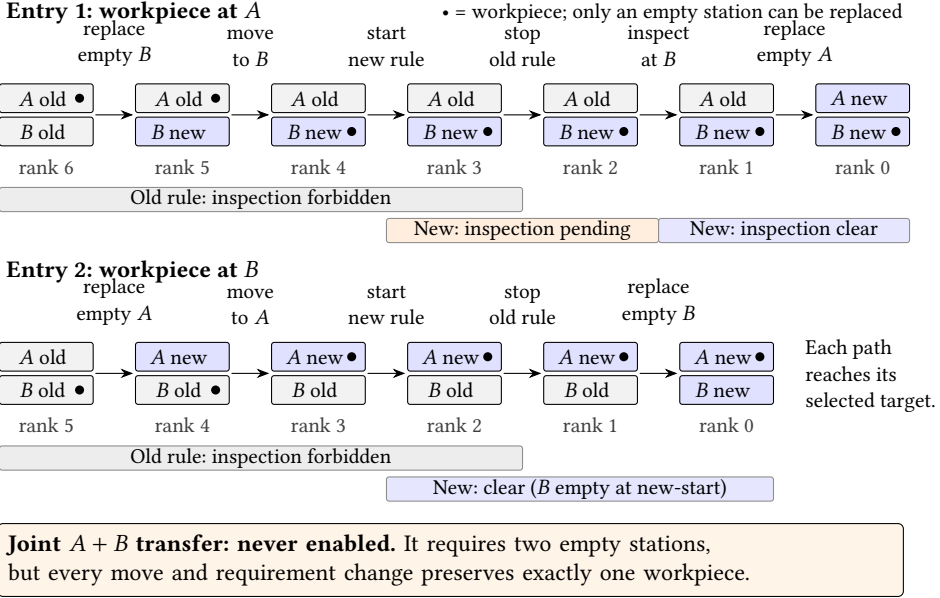
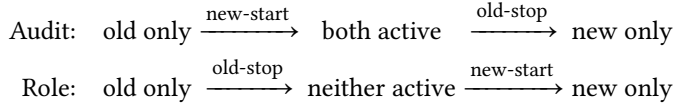


Fig. 1. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward its selected new target. The failed joint replacement uses the same endpoints and requirements.

2.2 Audit and Role Lifetimes

The constructed *Policy(2)* example has two old/new requirement pairs: audit and access role. Audit coverage must persist, while old and new roles must never overlap, requiring opposite boundary orders:



The role gap is declared by the interval requirement, rather than forced by access rules alone. The new audit monitor starts unlogged and requires a log before service. Each endpoint alternates its own log and service, permitting only its own access role. With an identity transfer, the saved policy obeys these opposing orders and reaches a compatible new endpoint from both old states in at most six events.

Merging starts and merging stops makes the first start overlap roles and the first stop remove audit coverage. Ordinary events and transfer cannot change the active rules, so neither entry has a completing merged policy. TA C gives the complete finite input, policy and losing closure.

2.3 Existing Update Interfaces

Table 1 contrasts native inputs and outputs. DUCS ensures eventual stop, start and reconfiguration after triggering, and enforces new goals from new-start; its transition requirement *T* constrains one global stop/start pair, including overlap, gaps and their duties [31, Def. 4.1]. FG-DUCS gives each requirement its own boundaries and initializer. GR(1) updating bounds switching under new-environment safety, then runs its synthesized controller unchanged; subsequent liveness assumes

Table 1. Native inputs and generated results: DUCS [31, Defs. 4.1–4.3, Sec. 5], GR(1) updating [2, Def. 3.1, Sec. 4], dynamic feature configuration [36, Secs. 4–7], and FG-DUCS.

Interface	Supplied inputs	Generated structure and output
DUCS	Old controller; old/new environments and goals; global transfer relation; transition requirements	Update environment and goal; controller and entry map covering every old-controller state.
GR(1) updating	Old/new GR(1) specifications and switching condition	New controller, bridge and switching bookkeeping.
Dynamic features	Feature/component models; individual and simultaneous changes; behavioral and configuration requirements	One safe, controllable, nonblocking, maximally permissive supervisor across configurations.
FG-DUCS	Fixed components/controllers; component transfers; monitors, initializers and precedence; allowed handover states	Mixed-version and boundary game; policy, rank and handover map for every old entry and permitted outcome.

environment justice [2, Def. 3.1, Sec. 4]. Dynamic-feature nonblocking requires a marked continuation from every reachable state [36, Sec. 7]. FG-DUCS requires every maximal update run to finish at the selected supplied endpoint (S5). On-the-fly DUCS (O-DUCS) separates update/post-update analysis and tries update events first [17, Secs. 2–3]; the archived fork’s on-the-fly branch connects to a synthesized new controller (S4.8).

Cell through native interfaces. DUCS permits staged reconfiguration inside its supplied new environment E' [31, Sec. 4.2]. TA I–J give chosen Cell encodings that hold the same eight version/holder configurations, two transfers and two requirement boundaries fixed (Table A4): our DUCS encoding adds an eventual handover event to T , beyond the published safety- T implementation [31, Sec. 5.3], and with its explicit post-handover graph also preserves endpoint traces; GR(1) may restrict endpoint edges when synthesizing its new controller. The native constructions supply their stage/guard equations, whereas FG-DUCS generates the operational structure from local inputs while still requiring monitors and initializers. These controllable, deterministic witnesses establish neither a general translation nor a cost advantage (S5).

3 Defining an Update Contract

The modeler supplies local behavior, transfers, monitored duties with activation states, fixed controllers and precedence; FG-DUCS generates the mixed configurations, commands, guards and handover tests of a global game, in which synthesis selects controllable choices while retaining every UC event and outcome.

Scope and premises. Five premises define this maintenance setting. **(i) Service may pause:** update control replaces old scheduling and preserves only declared monitors, not old liveness; duties needed throughout belong in interval monitors. **(ii) Global UC quiescence:** commands and handover require no enabled UC event in the active-component product (Equation (3)). This differs from transaction-based quiescence [20] and tranquility [38]. **(iii) No fairness or command priority:** persistent UC activity can prevent completion. **(iv) Event-count guarantee:** completion has no elapsed-time bound. **(v) Execution conformance:** observations and commands must follow the model (A1–A4, Section 4.3).

3.1 Supplied Choices

Finite transition systems and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{uc} \uplus \Sigma_c$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [24]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing error states F_r for violating prefixes [1, 21]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* ι_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed. Section 4.2 defines history-based and activation-scoped interpretations. For $r \in R^{\text{upd}}$, $\text{dom } \iota_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i \Sigma_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} \iota_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. An eligible contract satisfies the finiteness, typing, totality and non-error conditions above; dependencies form a strict partial order, and endpoint controller alphabets cover environment labels. Eligibility is structural.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i^v}$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active

until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states, installs interval initializers, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The update policy replaces C^{old} and its scheduling restrictions. Declared old monitors remain until their stops; undeclared objectives, including old liveness, are not preserved. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage forbids waiting for a favorable old state.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

Thus every command waits for global *uncontrollable quiescence*; fine-grained refers to the units of transfer and requirement change, not to the region that must become quiescent. Removing optional controls at UC-enabled states preserves winning; allowing commands alongside UC therefore adds no winners while handover remains quiescent (Lemma A.1). Command priority or fairness would define a different problem.

Ordinary controls may coexist with UC. The following rules define Post ; unmentioned coordinates stay unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } t_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = t_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals.* An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

The goal is a quiescent handover boundary, not merely completion of the command list. At a goal, interval obligations end and matched new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. UC cannot compete; compatible targets may have different continuations.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_\sigma(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_\sigma(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts events and assumes no fairness. Winning policies exclude controllable self-loops during update; a reached UC self-loop makes a state losing. Ordinary actions can progress, as Cell's moves and inspection do.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_\sigma(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_\sigma(q)$.

The FG-DUCS problem. Decide whether one policy safely completes the eligible game from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A *local update history* records an old entry and event/outcome sequence of the supplied operations (TA A); the proposition establishes state sufficiency, and Section 4.2 addresses requirement interpretation.

PROPOSITION 3.1 (CONTRACT/GAME CORRESPONDENCE). *For an eligible contract, histories with the same component tuple, active monitor memories and unexecuted commands admit the same next events and outcomes, safety status and handover targets. Folding histories into these states gives a label- and outcome-preserving correspondence with game paths. A history-dependent policy satisfying the source obligations exists exactly when a winning game policy exists.*

PROOF OUTLINE. Every source operation folds to exactly its game successors, preserving labels, outcomes and safety. Induction lifts paths and policies both ways; the finite reachability attractor supplies a state-based winner (full proof: TA A). $\square$

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned winning-policy certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions, without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies. TA H characterizes policy reuse under changed endpoints, including a saved production-cell instance named PC2.

Why matching and reconnection both matter. Cell’s prefix $\rho_B, m_B, \text{start}, \text{stop}, \rho_A$ finishes every command with inspection pending; physical matching alone would admit the clear target z_B , whose m_A violates that duty, so monitor matching excludes the reset. Conversely, disabling controllable loop b in a one-state endpoint with loops a, b and goal *true* preserves legality, deadlock freedom and safety [31, Defs. 3.9–3.10, 4.3] but loses b -continuations; FG-DUCS requires a supplied controller’s continuations, which safety alone does not, without excluding richer DUCS encodings.

4 What a Returned Policy Establishes

A returned policy is a certificate for the supplied game: from every old entry and under every modeled outcome it keeps every active monitor safe, reaches a quiescent handover after finitely many events without fairness, and restores the selected endpoint’s continuation (Theorem 4.1; Proposition 3.2). Whether it also establishes the modeler’s requirement or the platform’s execution depends on two further premises: the monitor and its initial state must express the intended duty, and observations and commands must follow the model. Table 2 separates these layers. OLD, NEW and UPD denote old, new and update-interval requirements.

Table 2. What a winning policy establishes for the supplied game, and the premises and checks for interpreting its requirements or execution.

Scope	Result and needed premise	Evidence and remaining scope
Game (all winners)	OLD monitors stay safe until stop, NEW monitors from start and UPD monitors through the goal. All entries/outcomes complete finitely and preserve the selected endpoint continuation.	Finite-game checks, six independent FSP fixtures and saved certificate summaries; successor generation is shared in the main suite.
NEW invariant from start	Truth at activation and while active, with faithful monitors and matching initialization/observations (Section 4.2).	900 accepted lookup cells meet the reference criterion; 21 actual starts of two saved Railcab crossing-control policies are checked. General activation and fallback behavior remain premises.
Prior-history duties	Continuations preserve declared history duties if residual inclusion holds and the scope contains actual histories.	Saved residual inclusions. One PC2 activation tuple has a safe-history witness; other tuples remain unchecked.
UPD from update entry	Interval duties hold from admission with correct boundary-state initialization and observations.	135 interval occurrences checked under reference observations; boundary-observation conformance remains a premise.
Execution	Observed events reach handover within the entry rank under A1–A4 (Section 4.3).	Platform execution is unvalidated; the bound counts events.

4.1 Synthesizing and Checking a Policy

The solver applies established strong-planning and on-the-fly reachability principles [6, 8, 9, 37]:

- (1) Discover all entries. Mark discovered goals winning with rank zero and matching end-points.
- (2) Expand a safe non-goal: query all UC successors or, if quiescent, all outcomes of the next enabled control. Discover each successor; unsafe states stay losing.
- (3) Repeatedly promote a safe UC-enabled state when every UC successor wins, or a safe quiescent state when some enabled control has only winning successors. Retain these choices and set its rank above every retained successor rank.
- (4) Return a policy once all entries win. Otherwise expand remaining discovered states and retry unresolved quiescent states with their next controls. If neither remains possible, return the nonwinning region.

TA A.1 gives the complete pseudocode (Algorithm 1), including cache and queue bookkeeping. Exploration may reach the whole game.

THEOREM 4.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), the lazy procedure above (Algorithm 1 in the TA) terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF IDEA. Promotion preserves complete successor sets of strictly smaller rank, giving safety and finite completion. Finite exhaustive retry ensures termination. At failure, every UC-enabled nonwinner has a nonwinning outcome, and every enabled control at a quiescent nonwinner has one. This region prevents completion (full proof: TA A). $\square$

Ranked policies are checked for entry coverage, enabled choices, all-outcome closure, decreasing ranks, safety and matching goals, and losing-region checks verify the counter-outcomes above; both are relative to the generated successors and endpoint predicates [18] (TA A.2).

4.2 Giving Requirements Their Intended Lifetimes

The game interprets a requirement through its monitor and the state installed at activation. Two intended lifetimes occur in the inputs: a duty that depends on the history before the start, and an invariant that begins when its requirement starts.

A duty installed at activation. Cell’s initializer sets the owed-inspection bit exactly when B holds the workpiece. Event m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Monitor states c, d encode bits 0,1 with identical transitions, including rejection. Matching initialization and induction over events therefore prove exact enforcement of this duty rule. A start-only history reference would also admit clear initialization while B holds, omitting this duty (TA B.1). Prior-history exactness alone therefore does not select the intended meaning.

Invariants that begin at activation. For an invariant $\Box\phi$ intended to begin at activation, let v be the post-start observer values and $d^*(v, u)$ their values after event word u . Its permitted continuations are

$$\text{ACT}_\phi(v) = \{w \mid \forall u \preceq w : \phi(d^*(v, u))\},$$

including the empty prefix: the invariant must hold at activation itself. A matching initializer selects monitor state m with continuation language $L_M(m) = \text{ACT}_\phi(v)$. For a faithful invariant monitor, safe reachability of (m, v) in the unrestricted monitor–observer product suffices (TA F.1).

The witnessing reference word establishes the continuation language and need not be an executable update history; actual initialization and subsequent observations must realize that language, and commands that preserve physical observer values require the compatibility argument in TA F.2. Truth of φ alone does not ensure a lookup entry, and undefined entries or fallback initializers are outside this criterion.

Other declared lifetimes. An interval monitor begins at update entry. A requirement retaining duties from prior histories needs *residual inclusion*: every continuation accepted by its initialized monitor must be allowed after every history in the declared scope. That scope must contain the actual histories, since an empty scope makes inclusion vacuous. TA E gives the full criteria and saved checks; synthesis does not choose this interpretation for the modeler.

An active requirement during ordinary behavior. For the inherited GSM message-processing input with interval guards, an analytical policy starts the new rule $\Box\neg\text{decode}.2$, then permits $\text{decode}.3$ and output before handover. It rejects $\text{decode}.2$ and covers every entry within twelve events (TA F). This illustrates start-scoped enforcement; it is not a saved solver output.

4.3 Executing the Policy

Execution requires four premises (TA G):

A1: Observations. Endpoint observations preserve component/monitor states and same-labeled edges.

A2: Event progress. Control disables every unselected controllable, retains every UC event and outcome, and eventually observes a next event and successor.

A3: Commands. Commands start quiescently and finish atomically, finitely and without failure, reporting a modeled successor.

A4: Boundaries. Entry copies the old state; quiescent handover installs the selected compatible state. Both are finite, failure-free and linearizable [16].

Under A1–A4, execution reaches handover within the entry rank (Theorem G.1). A2 adds no fairness preference or time bound. History-based duties also require the declared scope to contain actual activation histories. Internal divergence and unobservable outcomes violate these premises.

5 How Granularity Changes Feasibility

Granularity is part of the supplied contract: a batch removes the states between its member operations, which may be indispensable or harmful. We first identify obstructions to a specified merge, then give local conditions under which a merged policy can be implemented by separate commands.

5.1 The Compared Contracts

Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ permits every target in $\prod_{i \in B} g_i(s_i)$ in one step. A stop block removes all its monitors before observation; a start block initializes all its monitors afterward, at the unchanged component tuple. Each monitor active throughout a block applies every member-label transition once. These transitions must commute pairwise on every monitor state, making their order irrelevant. Mapped precedence must remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply.

5.2 Obstructions and Capacity

LEMMA 5.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed*

under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled; Policy retains its initial active bits until either grouped boundary violates the interval; PC2-Rolling’s enabled joint transfer violates availability (Section 6.2.1). These results concern specified merges, not native DUCS expressiveness. Readiness gates motivate the constructed Rolling contract [29, 35]. It abstracts capacity during paused maintenance, rather than modeling those platforms’ continuous-service policies. It starts with n ready replicas, no spares, and an interval floor of m ready replicas. A transfer boots its replicas; each becomes ready after one UC event, with no waiting loop. Service self-loops are controllable. The all-new, ready target restores capacity; service may pause during the update.

PROPOSITION 5.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

5.3 When a Batch Can Be Serialized

Finer commands need not preserve a merged winner. Consider two one-state components sharing UC labels x, y . Component 1 permits only y before transfer and only x afterward; component 2 has the opposite loops. Neither label synchronizes in the all-old or all-new product, but either mixed product has a UC loop. Separate identity transfers therefore lose, whereas joint transfer reaches the target at rank one. A controllable loop keeps both endpoints deadlock-free. TA D.2–D.3 give the complete contract and a second counterexample in which the intervening UC event terminates but destroys target compatibility.

For component i , let A_i^v be its UC alphabet, $E_i^v(s)$ its enabled UC labels and $B_i^v(s) = A_i^v \setminus E_i^v(s)$ its blocked UC labels. A blocked participant prevents synchronization; an absent label does not. Require

$$E_i^{\text{new}}(s') \subseteq E_i^{\text{old}}(s), \quad B_i^{\text{old}}(s) \subseteq B_i^{\text{new}}(s') \quad \text{for every } (s, s') \in g_i.$$

The transfer must offer no new UC label and remove no blocker; the table lists the allowed changes per label, where “absent” means outside the local alphabet, hence not participating in synchronization.

Old status \ New status	Absent	Enabled	Blocked
Absent	yes	no	yes
Enabled	yes	yes	yes
Blocked	no	no	yes

Removing a blocked label can thus enable a global event even though it enables no local transition. With equal alphabets, the criterion reduces to enabledness inclusion.

PROPOSITION 5.3 (CONTEXT-INDEPENDENT QUIESCENCE). *For a fixed local transfer pair, the two inclusions above hold iff that replacement preserves UC quiescence in every finite synchronous context whose other states stay fixed.*

PROOF OUTLINE. The inclusions prevent a new globally enabled label. Conversely, a context can expose either a newly enabled label or a removed blocker, while blocking all other UC labels. TA, Proposition D.1, gives both constructions. $\square$

This equivalence concerns quiescence alone. Whole-policy serialization also needs the Cartesian outcomes, commuting monitor effects and valid precedence of Section 5.1.

PROPOSITION 5.4 (SERIAL TRANSFER PRESERVATION). *For transfer-only merging under the comparison rules of Section 5.1, if every transfer satisfies the local UC criterion above, a merged winner implies a fine winner. A finite-memory serial expansion preserves the merged policy’s selected endpoint continuations.*

PROOF. Fix an order for each batch and disable ordinary controls during its expansion. *Quiescence and enablement.* Each transfer preserves quiescence with the other components fixed, including earlier replacements. Thus every partial batch is quiescent. Unexecuted transfers retain their source coordinates and nonempty images; irreflexive mapped precedence forbids dependencies within a block.

Monitor safety. Every partial outcome extends to a Cartesian batch outcome. Commutation yields the same final monitor states. An intermediate absorbing error would persist to the safe batch endpoint, a contradiction.

Completion and reconnection. Replacing ranked coarse steps by these finite sequences preserves completion and the selected endpoint. Each completed run gains exactly $|I| - b$ events, for b transfer blocks. The expansion uses finite memory; memoryless sufficiency gives a fine winner but need not preserve the merged endpoint selection (TA D.3). $\square$

The main solver does not automate this expansion; Section 6.2 checks a separate implementation on one restricted family. Threads, a constructed worker model with a bounded queue, passes the local test: its identity transfers between identical old/new graphs leave enabled and blocked sets unchanged. Rolling fails: its new booting state enables a UC readiness label absent in the old ready state. This is the table’s absent-to-enabled case, yet Rolling wins under its capacity threshold. Failure of the test leaves feasibility open (TA D.1).

PROPOSITION 5.5 (SERIAL TRANSFER AND BOUNDARY PRESERVATION). *Let $\mathcal{B}$ partition all commands under Section 5.1’s comparison rules. Suppose every transfer in a nonsingleton block satisfies the UC criterion; each start-block monitor self-loops on other member starts at every common admissible initialization tuple; and each safe stop-block member state stays safe on other member stops. Every merged winner then has a finite-memory fine expansion preserving its selected endpoint continuations. Each expanded run gains exactly $|\Sigma_{\text{upd}}| - |\mathcal{B}|$ events.*

PROOF OUTLINE. Boundaries leave components and quiescence unchanged. The conditions protect monitors newly started or awaiting removal; commutation and error absorption handle monitors active throughout. The checkpoint proof above applies to every block (full proof: TA D.4). $\square$

Boundary conditions protect members started or removed together: after splitting, one can observe another’s command. TA D.4 gives counterexamples without this protection.

6 Implementation and Evaluation

6.1 Design and Implementation

RQ1 examines granularity mechanisms on constructed contracts; RQ2 checks the implementation and selected requirement interpretations; RQ3 compares exploration costs within the same solver

on 27 adapted contracts. WIN and LOSS denote proved winning and losing games; timeout (TO) and incomplete (Inc.) denote unresolved results.

Each inherited input has three supplied configurations: Base adds no interval requirement; R1 protects selected gap actions; R2 requires selected boundaries before transfers. TA M gives events and sizes. A production-cell case separately tests availability. The archived fork’s generalized-reactivity DUCS backend (Legacy) solves a different subsequent-control problem and supplies separate reference results.

Implementation. Our Modal Transition System Analyser (MTSA) extension [11] checks certificates [27]; Link checks connected controllers, entries and targets. An independent oracle parses restricted Finite State Processes (FSP) [24]. Explicit finite inputs save losing membership; FSP benchmarks report statistics.

Generative tool use. Generative artificial intelligence (AI) assisted contract and example design, proofs, the Java solver and frontend, Python checkers, experiment drivers and execution, analysis and editing. Validation used certificates, exhaustive algorithm checks, source and automaton comparisons for initialization, saved-output table checks and primary sources for citations (TA Q).

Constructing the benchmark inputs. Nine inputs adapt eight DUCS sources; PC1 and PC2 share a parent. They have 1–5 components, 4–131 old entries and 3–26 update commands (TA, Table A6). Adaptations make production-cell tool results UC while the inherited arrival cycle stays state-gated, so withholding controllable work and output lets pending results, resets and arrivals finish without a total arrival budget; they shorten Railcab transfers while retaining both outcomes, remove PowerPlant’s duplicate initial-environment transfer declaration, and change safety and ordering. Their necessity for completion and equivalence to the source problems are unestablished (TA P).

MTSA synthesizes fixed endpoints from FSP and prepares monitors and initialization tables before search. Interval requirements use a constant post-entry state; physical observers survive commands and transfers. The Cell_n family extends Cell to n stations with holder-dependent initialization. S3–S4 detail these paths and their preparation costs.

Exploration variants. All variants share the game and checks. Lazy queries outcomes for one controllable event at a time; Eager queries outcomes for all controllables. Direct-Full first constructs the reachable graph; Full+UC additionally prunes controls at UC states. Update-first prioritizes update commands [17]. Query/retry orders affect exploration, not execution (TA L).

Design and measures. RQ3 uses one Windows/Xeon host, a 64 GiB heap and a 1,200 s Java Virtual Machine (JVM) cap: 27×4 game cells plus 27 Legacy references. Medians require five completed runs, unresolved trials remain capped, solver time includes checks, JVM time includes preparation, endpoints, startup and output, and Legacy uses its native timer. Mac pilots are excluded (TA Q, P.4; S4).

6.2 RQ1: Does Granularity Change Feasibility?

Answer. The measured suite separates 33/55 fine/merged pairs; none of 54 inherited comparisons changes its decision. Ten saved policies check serialization on one restricted family. These are mechanism checks, not estimates of prevalence.

Constructed mechanisms. Table 3 separates the mechanisms and controls. All measured separations are fine-WIN/merged-LOSS; the reverse direction is analytical (Section 5.3). They have three direct causes: capacity loss (Rolling, Canary and Rolling+Audit), opposed boundary orders (Policy), and unavailable joint transfer (DB-Rolling at $m = 2$, where some replica is always primary

Table 3. Granularity mechanisms, parameter sweeps and controls. Each pair compares fine and merged contracts of one model: Policy merges boundaries, other rows merge transfers. Counts are comparisons, not independent applications. The main total excludes Cell.

Family / control	Mechanism and parameters	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
<i>Constructed families and parameter sweeps</i>						
Rolling	Readiness during startup. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	Failure reports and recovery. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	Audit overlap; role exclusion. 2 rule pairs	1	1	0	0	0
DB-Rolling	Database (DB) role-gated replacement. $n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	Readiness and audit overlap. $n = 2, m = 1$; 1 audit pair	1	1	0	0	0
<i>Quiescence control</i>						
Threads (control)	Backpressure or persistent arrival. $n = 2 \dots 6$; queue 1,2	20	0	10	10	0
<i>Source-derived attempt</i>						
PC2-Rolling	Availability during calibration. 2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	Empty-station transfer guards. $n = 2 \dots 10$	9	9	0	0	0

Outcomes read fine / merged; Inc. is unresolved. PC2's merged solver remains a timeout (TO), not LOSS. Rolling and Canary share a capacity obstruction: their sweeps supply 30 of the 33 WIN/LOSS pairs. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. Threads varies queue capacity 1,2 under both arrival regimes. Rolling+Audit's boundary merge remains WIN and adds no pair here.

or an election candidate). The 33 separated pairs comprise 31 capacity cases and one of each other cause; 30 are Rolling/Canary parameter sweeps. Threads supplies arrival controls: backpressure disables arrivals at capacity, while an arrival self-loop defeats both granularities. A temporal gap between arrivals is insufficient; the event must be disabled. TA D.1 gives all models.

6.2.1 A Source-Derived Calibration Case. PC2-Rolling supplies two arm models and transfer maps, 12 old and 13 new requirements, an interval keeping one arm operational, and fixed controllers. From these, FG-DUCS generates 25 requirement boundaries and two transfers, without a prescribed order. Requests are controllable and completions UC; transfers enter calibration and preserve initializer observers. Erasing calibration recovers the operational process, not whole-problem equivalence (TA K).

The policy calibrates each arm in turn, covering all 126 entries with at least 1 arm outside calibration. Every outcome decreases rank from at most 37; Link checks reconnection. Thirteen source invariants hold from their starts through the remaining start-only suffixes. This checks reached

tuples up to handover; Section 4.2 gives GSM’s analytical example with ordinary actions after activation.

Why the fixed merge loses. Apply Lemma 5.1 with B containing only the joint transfer and R the safe states reachable before it. Old plants lack calibration actions; all other steps preserve both readiness bits. Thus R is closed before B , whose every outcome calibrates both arms and violates the interval. The merged solver and fine Direct-Full nevertheless time out at 7200 s/200 GiB. Restricted-transfer controls retain both-LOSS and incomplete cases (TA P; S1–S4).

Inherited inputs: no observed change. The 54 comparisons are nine inherited inputs $\times$ two configurations (Base/R1) $\times$ three merges (transfers, boundaries, both). None changes its decision; merging reduces or preserves the state count. R2 names individual update events and is excluded. Their total physical transfer relations and absent post-transfer startup/calibration phases exclude those particular separation mechanisms (S4.1). The null result remains empirical: total transfers do not imply safe merging, because requirements may reject the joint successor, as PC2-Rolling shows. The local criterion covers 12/18 Base/R1 transfer-only pairs in six models: 11 WIN/WIN and one LOSS/LOSS (TA, Table A10). PC1/PC2 remove blocked UC labels and Railcab enables one; their other six pairs are empirically WIN/WIN without this guarantee. No inherited policy was converted.

Expanding saved batch policies. An offline transformation expands all ten saved Threads back-pressure policies ($n = 2\text{--}6$, queue capacity 1 or 2). Checks cover 620 entry occurrences, every UC action and selected outcome, decreasing ranks and the same endpoint; every case passes with exactly $n - 1$ added events per completed path. This restricted family has identical OLD/NEW behavior, idle identity transfers, no requirement monitors and no UC interleaving in added phases. It validates neither a general converter nor speedup (all cases: TA R).

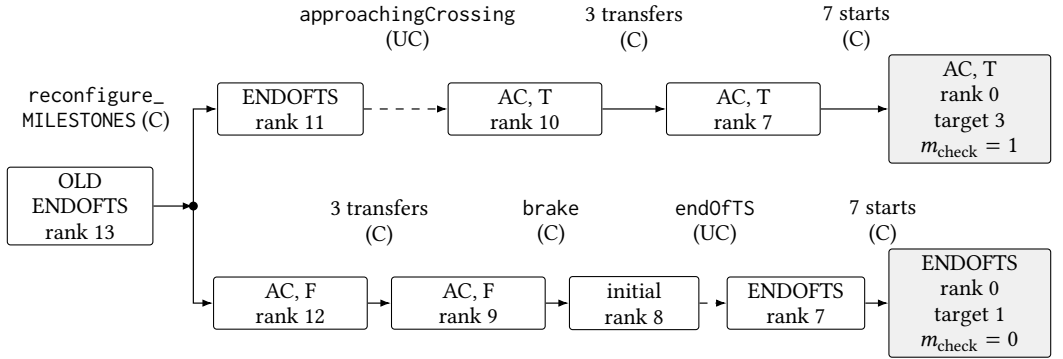


Fig. 2. Both outcomes of one Railcab/Base transfer after five old stops and before any new start (Base has no interval monitor). AC is approaching-crossing, T/F records whether that event occurred since endOfTS, boxes give milestone state and rank, targets identify fixed new-controller states, and m_{check} is the check-status monitor state. Only single-path sequences of three transfers or seven starts are contracted; every branch of this 26-state, 25-edge fragment is retained.

6.3 RQ2: What Do the Checks Establish?

Answer. The checks support the game and selected requirement interpretations in Table 2, not platform execution; TA P.1 gives all validation populations.

Interpreting a saved WIN. Figure 2 shows a saved Railcab Base transfer whose two outcomes need different continuations and endpoints. The check-status invariant permits checking only after approach and before the last emergency-brake point. Its start installs state 1 on the upper branch and 0 on the lower; clearing both would forget the upper approach. Decreasing ranks and these activation states illustrate the game and NEW-invariant rows of Table 2.

After handover, target 3 enables checkCrossingStatus and requestEnter, retaining both UC responses, working and notWorking. Target 1 reaches that checking state through idle_c then approachingCrossing. These are supplied endpoint continuations.

Two saved Base/R1 policies cover all 22 entries and pass certificate/Link checks. Replay checks all 21 NEW starts and every ordinary endpoint continuation: installed states match the lookup, observations agree, and all seven NEW invariants hold from their starts. This assumes 22 exported old-entry vectors per policy; after the first start the update suffix contains only starts (TA F.3). Base has 1–4 ordinary gap events; R1 has none and retains seven guards (TA P.5; S4.4.3). These are saved-policy properties, not runtime evidence; WIN alone imposes no gap duty.

Checking the generated game. Python reconstructs explicit successors for the 92-job suite, and all winning jobs pass Link, which checks the connected controllers. Deliberately invalid cases omit events, deadlock or suppress UC behavior. This suite shares frontend, endpoint and goal code. Six additional monitor-free fixtures independently reconstruct FSP endpoints, games and goals (S4.2).

A losing contract. Exhaustive closure shows Industry/R1 loses at 42/131 entries under every order: validating either of two components violates new requirements; another component may reject forever, or deadlock quiescently if safe validation is withheld. Native DUCS wins its different subsequent-control problem (S4.4.2, S4.8).

Checking requirement lifetimes. In PC2-Rolling (Section 6.2.1), every maximal run of the saved policy crosses the selected start, so all 126 entries activate the selected output requirement at one observer tuple. In the saved graphs, 39 admit violation-free histories to it; 87 admit none. Prepending a violating cycle preserves that tuple, so all 126 fail whole-history preservation while retaining the start-scoped guarantee (TA K).

Reference checks cover 46 NEW requirement positions and 135 interval occurrences across nine inputs and three variants (Table 2; TA E, F.1), not executions. Command-observation analysis preserves accepted states but finds reference-safe rejected cells, exposing extra activation restrictions; completeness remains relative to the initializer domain (TA F.2). Empty observer selections or absent mappings use fallback; missing lookup signatures exclude activation (S3.1). General decoding and fallback use across all 25 WIN contracts remain unverified; the Railcab replay covers actual keys in two exports.

6.4 RQ3: Does Lazy Exploration Reduce Cost?

Answer. Lazy decides 26 contracts versus Eager’s 17 under identical UC pruning. All 14 completed Direct-Full pairs agree (plots: TA N); Lazy uses fewer states/queries with additional pruning. Update-first is faster in 8/24 pairs but adds timeouts. JVM ratios include preparation, sometimes exceeding solver time (Table 4, Figure 3; S4.3).

Lazy proves Industry/R1 losing; PC2/R2 times out at the main cap, at 3,600 s/64 GiB, and in the earlier 7,200 s/200 GiB series. The limiting resource is unknown (S4).

Table 4. All 27 contracts per method. TO is one capped trial. Ratios compare heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Faster / pairs	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—
Eager	17/0/10	0 / 17	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	8 / 24	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	0 / 14	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains interruption and same-setting timeout (S4).

Comparator/Lazy ratios: median [min–max] of paired five-run medians; unresolved costs excluded. Faster / pairs counts lower comparator solver medians. JVM includes frontend, startup, endpoints and output; solver includes checking.

Legacy’s unmatched references retain 9 WIN, 13 out-of-memory (OOM) failures and 5 unmeasured snapshot-cap cases (TA P).

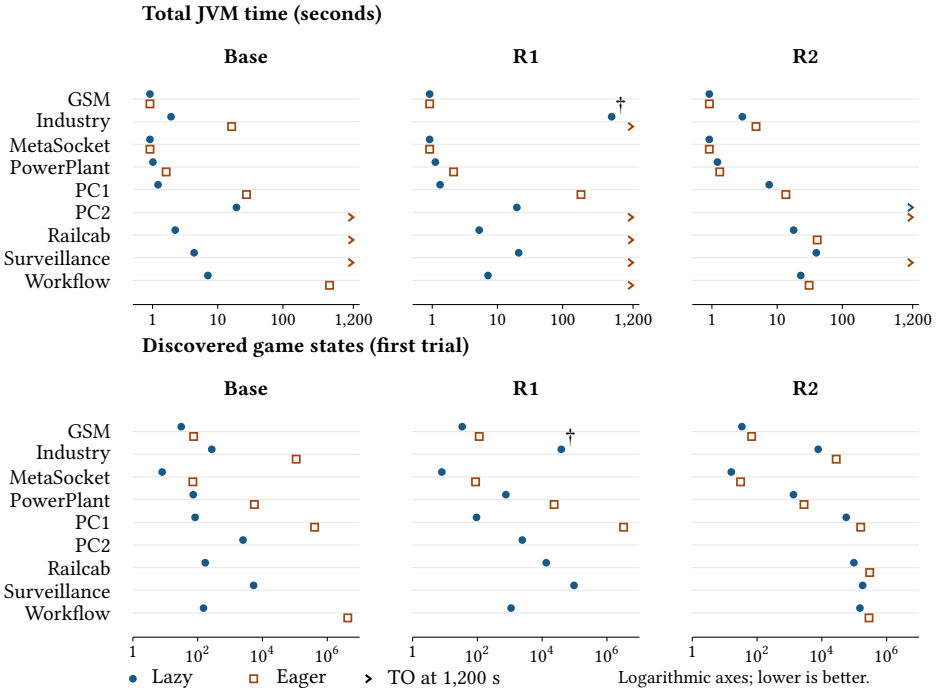


Fig. 3. All 27 contracts under the same UC pruning. Completed times are five-run medians with min–max bars, including preparation, solving, checking and other JVM work. TO arrows mark one capped attempt, not a median or a solver-time bound; no state count is imputed. [†] is Industry/R1 LOSS; other completed points are WIN.

Cost and scaling scope. Full+UC mixes builds, hosts and repetitions, and an unpruned on-the-fly variant is unimplemented, so pruning and interval effects are not isolated. All methods finish the independent-transfer family; Eager/Update-first sometimes use less peak resident memory. Travel couples an agency with amenities and retains timeouts and memory failures, including expanded-budget failures excluded from medians (S3–S4).

7 Related Work

Update environments and synthesis. Global state-preservation transitions generate update environments [39, Sec. V]; DUCS permits a partial global transfer relation R , without UC quiescence [31, Sec. 4.2, Defs. 3.7, 5.1]. Directed controller synthesis (DCS) uses lazy products, UC priority and backward propagation for eventual goals on every trace [9, Sec. IV]; nonblocking on-the-fly DCS requires a marked extension of each finite trace [10, Def. 4].

Requirements and configuration changes. Temporal adaptation, scenario updatability and Live Synthesis handle changing obligations [13, 14, 25, 40]. FG-DUCS uses scoped requirements and residual reasoning [12, 34]. Dynamic feature configuration supports individual and joint changes, feature-dependent requirements and retained/reset state; its joint feature swap avoids invalid intermediate configurations [36, Secs. 4–7].

Switching and operational updates. State attraction ensures bounded reachability of a startup set from every request state within a task language [32, Def. 2, Alg. 1, Thm. 1]. Switched supervisory control provides safe, deadlock-free switching and reinitialization between supplied safe/live supervisors; its full problem requires every finite trace to extend to a switching opportunity [28, Problem 1, Sec. 3.3]. FG-DUCS generates the mixed system from local transfers and requirement lifetimes, with explicit grouping.

Granularity and atomicity. Lipton preserves final states through atomic reduction under commutation and nonblocking [23, Sec. 3]. Partial-order methods omit redundant interleavings [15, Secs. 4–5]. Our expansion preserves winning strategies and selected endpoints through intermediate quiescence, transfer outcomes and monitor observations. Network-update synthesis distinguishes switch/rule granularities, preserving linear temporal logic (LTL) properties while reaching prescribed final packet traces [26, Secs. 3.1–3.2, 4.3]. FG-DUCS handles requirement activation states and environment-chosen transfer outcomes, completing from every entry without fairness; a general translation is unestablished.

8 Threats to Validity

Internal validity. Certificates, exhaustive checks and independent restricted-FSP parsing mitigate correlated faults from shared authorship, AI and code. Direct-Full is in-house; external solvers and randomized orders are untested. Five-run medians exclude extended trials. Null results, losses, timeouts, memory failures, interruptions and faster or smaller alternatives are retained (S4). Memory sampling can miss peaks and cannot isolate heap use or bottlenecks.

External validity. One host and related models limit generalization; repeats are not independent applications. Inherited inputs lack explicit precedence; only Railcab transfers branch. Constructed cases assume finite startup, reporting, recovery and election; selection may favor separations. Deployment prevalence, model-adaptation effects and platform conformance remain unestablished. Activation analysis covers only the stated saved cases; failure repair is excluded. The supplementary exploration bound depends on the default query order (TA L).

9 Conclusion

FG-DUCS generates an update game between verified controllers from local transfers, requirement lifetimes and initial obligations. Winners finish safely from every entry under all outcomes without fairness, restoring selected endpoint continuations. Serialization conditions, obstructions and a capacity threshold delimit granularity. Constructed separations coexist with 54 unchanged inherited comparisons; synthesis decides 26/27 adapted contracts. Commands require global UC quiescence; execution conformance, continuous service and elapsed-time bounds remain unestablished.

Data Availability

We created the anonymous public replication package for this research [4]. Table 5 identifies the matching manuscript, supplement and source bundle in the published revision. The implementation and experimental results are unchanged from the earlier archive. The source also includes analysis of saved inputs and policies; these checks require no new performance experiments. Dependencies come from official upstream sources.

Table 5. Locations and correspondence of the materials.

Material	Location and contents
Public replication package	Cited release fgducs-c143-20261004 : implementation, all 27 application contracts, scripts, saved results and the matching documents below. Its README maps paper claims to proofs, evidence and reproduction commands.
Matching paper and supplement	paper/main.pdf and paper/supplementary_material.pdf . Supplement Part I is Technical Appendix A–R; Part II is S1–S5. A reading guide, bookmarks and continuous page numbers connect the proofs and evidence to this paper.
Matching manuscript source and added analyses	paper/manuscript_source.zip includes the manuscript and three saved-evidence analyses. Their scripts, copied inputs and results are also browsable under paper/source/evidence/ : Railcab activation , command observations , and Threads serialization .

The retained evidence snapshot is also available at <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-invariants-20261002>.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. Full text, authors' research-group repository. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Public anonymous replication package prepared for this research, including source code and saved evidence; matching manuscript and supplements at tag fgducs-c143-20261004. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c143-20261004>
- [5] Fabienne Boyer, Olivier Gruber, and Damien Pous. 2013. Robust Reconfigurations of Component Assemblies. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 13–22. doi:10.1109/ICSE.2013.6606547
- [6] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [7] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1

- [8] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [9] D. Ciolek, V. Braberman, N. D'Ippolito, and S. Uchitel. 2016. *Directed controller synthesis of discrete event systems*. Technical Report 1605.09772. arXiv. doi:10.48550/arXiv.1605.09772
- [10] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [11] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [12] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [13] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [14] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAMS.2012.6224401
- [15] Patrice Godefroid. 1991. Using Partial Orders to Improve Automatic Verification Methods. In *Computer-Aided Verification*. Lecture Notes in Computer Science, Vol. 531. Springer, 176–185. doi:10.1007/BFb0023731
- [16] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [17] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. KBSE2023-78; in Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/eng/>
- [18] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [19] Ting Jiao, Yongmei Gan, and Guochun Xiao. 2013. On the Reconfiguration of Flexible Manufacturing Workcells. In *2013 IEEE International Conference of IEEE Region 10 (TENCON 2013)*. IEEE, 1–4. doi:10.1109/TENCON.2013.6718470
- [20] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [21] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [22] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [23] Richard J. Lipton. 1975. Reduction: A Method of Proving Properties of Parallel Programs. *Commun. ACM* 18, 12 (1975), 717–721. doi:10.1145/361227.361234
- [24] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
- [25] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
- [26] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
- [27] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [28] Rômulo Meira-Góes and Stéphane Lafortune. 2020. Moving Target Defense based on Switched Supervisory Control: A New Technique for Mitigating Sensor Deception Attacks. *IFAC-PapersOnLine* 53, 4 (2020), 317–323. doi:10.1016/j.ifacol.2021.04.031

- [29] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [30] Leandro Nahabedian, Víctor Braberman, Nicolás D'Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056
- [31] Leandro Nahabedian, Víctor Braberman, Nicolás D'Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. *Accepted manuscript, Imperial College London repository*. doi:10.1109/TSE.2018.2876843
- [32] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [33] Spyros Reveliotis and Zhennan Fei. 2017. Invariant-Based Supervisory Control of Switched Discrete Event Systems. *IEEE Trans. Automat. Control* 62, 2 (2017), 921–927. doi:10.1109/TAC.2016.2560536
- [34] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [35] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [36] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [37] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [38] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D'Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [39] Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2020. Method for Low-Cost Environment Partitioning Modeling in Dynamic Update. In *Proceedings of the 2020 IEEE Third International Conference on Artificial Intelligence and Knowledge Engineering (AIKE '20)*. IEEE, Piscataway, NJ, USA, 183–187. doi:10.1109/AIKE48582.2020.00036
- [40] Ji Zhang and Betty H. C. Cheng. 2006. Using Temporal Logic to Specify Adaptive Program Semantics. *Journal of Systems and Software* 79, 10 (2006), 1361–1369. doi:10.1016/j.jss.2006.02.062
- [41] Ji Zhang, Betty H. C. Cheng, Zhenxiao Yang, and Philip K. McKinley. 2005. Enabling Safe Dynamic Component-Based Software Adaptation. In *Architecting Dependable Systems III*, Rogério de Lemos, Cristina Gacek, and Alexander Romanovsky (Eds.). Lecture Notes in Computer Science, Vol. 3549. Springer, Berlin, Heidelberg, 194–211. doi:10.1007/11556169_9